INTRODUCTION AUX BASES DE DONNÉES ET SQL

RESSOURCE R1.05

I. EL KORBI

ORGANISATION DU COURS

- Volume horaire
  - 7h de CM
  - 16h de TD (On utilisera plutôt le mode cours intégrés)
  - 22h de TP
- Modalités d'évaluation
  - QCMs
  - Ramassage des TPs
  - Contrôle final (sur table +TP)

INTRODUCTION GÉNÉRALE

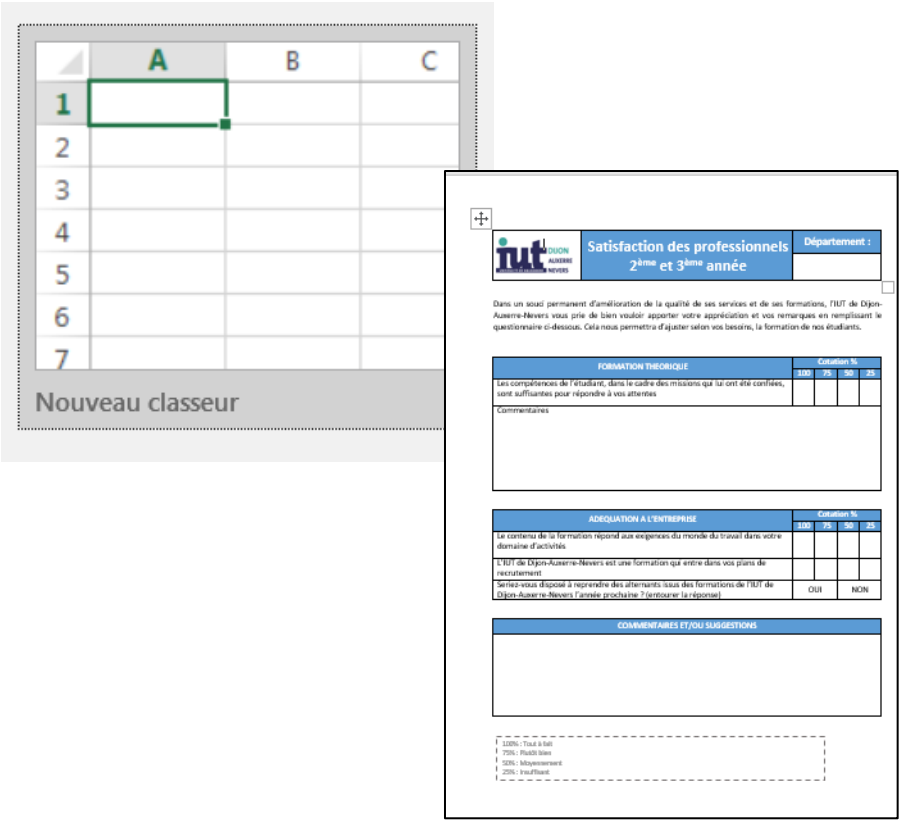


INTÉRÊT DES BASES DE DONNÉES ?

- **Données non informatisées**
  - Fiches papiers, stockage encombrant



- **Données informatisées**
  - Tableur Excel, fichier Word



Limites de l'utilisation des fichiers pour stocker les données

☹ Redondance des données
☹ Incohérence
☹ Exploitation difficile
☹ Maintenance fastidieuse

Solution:
Les bases de données

LES BASES DE DONNÉES

- **Définition 1:**
  - Une base de données (BD) est un ensemble structuré et cohérent de données enregistrées avec le minimum de redondance afin de satisfaire simultanément les besoins de plusieurs utilisateurs
- **Définition 2:**
  - Une base de données est une collection organisée d'informations structurées, généralement stockées électroniquement dans un système informatique.
- **Exemples:**
  - Annuaire téléphonique, dictionnaire, …
- Une base de données est généralement contrôlée par un système de gestion de bases de données (SGBD)



SYSTÈME DE GESTION DE BASES DE DONNÉES (SGBD)

- Logiciel ou ensemble de logiciels permettant une utilisation efficace de bases de données:
  - Définition des données (structuration)
  - Manipulation des données (insertion, mise à jour, suppression, interrogation)
  - Contrôle d'intégrité: cohérence des données
  - Gérer les accès concurrents: autorisation d'accès multiples, verrouillage en cas de modification
  - Confidentialité: privilèges d'accès
- Exemples: Oracle, MySQL, PostgreSQL, …





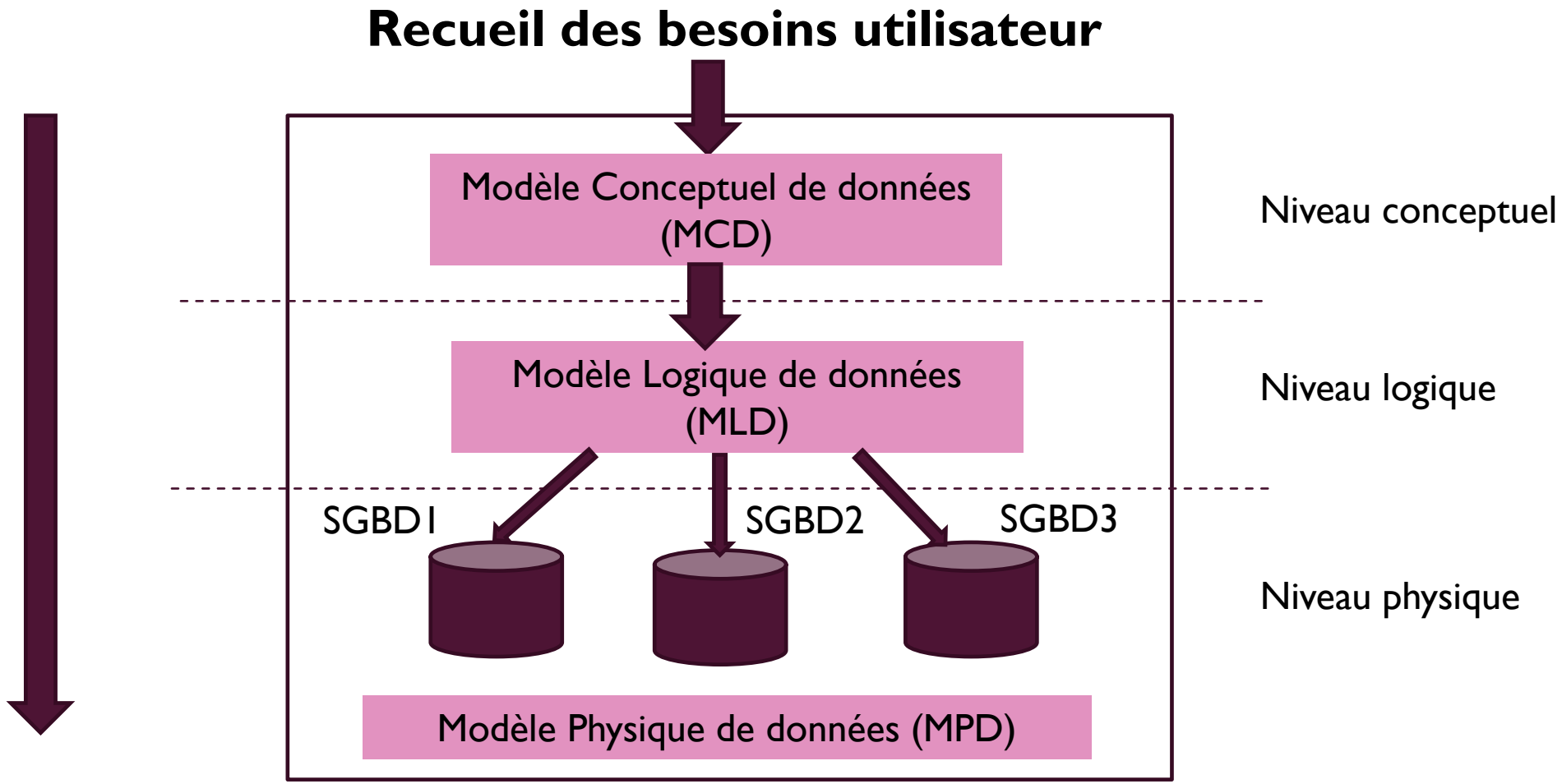




CHRONOLOGIE

- Avant l'informatique: informations sous forme de fiches (papier) classées par ordre alphabétique
- 1950-1965: organisation classique en fichiers (SGF)
- 965-1970: Système de gestion de bases de données (SGBD 1ère génération):
  arrangement des données dépendant des supports physiques
- 1970-1980: SGBD 2ème génération ou SGBD relationnel: basé sur les mathématiques et indépendant du support
- Années 80-90: SGBD 3ème génération: SGBD relationnel et orienté objet
- Années 2010: bases de données NoSQL

CONCEPTION D'UNE BASE DE DONNÉES (1)



CONCEPTION D'UNE BASE DE DONNÉES (2)

- Etapes:
  1. Analyse des besoins utilisateur
  2. Structuration des données: regrouper les données en « paquets » homogènes appelés « entités »
  3. Construction du modèle conceptuel de données (MCD) ou modèle entité entité-association
  4. Transformation du MCD en MLD (Modèle logique de données) ou modèle relationnel
  5. Implémentation dans un SGBD (modèle physique de données)

LE MODÈLE CONCEPTUEL DE DONNÉES (MCD) OU ENTITÉ-ASSOCIATION

CHAPITRE I

DÉFINITION D'UN MODÈLE CONCEPTUEL DE DONNÉES (MCD)

- La modélisation conceptuelle des données est la représentation de l'ensemble des données du système étudié sans tenir compte des aspects techniques de leur mise en œuvre.

- Le modèle conceptuel de données décrit la « sémantique » ou le sens des données et les rapports qu'ils ont les uns par rapport aux autres.

- Il permet de représenter de manière visuelle les liens entre les différentes données

ELABORATION DU DICTIONNAIRE DE DONNÉES

- **Etape 1:** Analyse et recensement de toutes les données à modéliser (documents papiers, fichiers, comptes rendus d'entretien oral)
- **Etape 2:** Elaboration du dictionnaire de données:
  - Il est formalisé par un tableau afin de caractériser chaque donnée (nom, description, type et taille)
  - Il est établi à partir:
    - Des interviews des acteurs impactés par le projet
    - De l'analyse des documents existants (bons de commande, devis, facture)
  - Types de données : numérique, alphanumérique, texte et date
  - Exemples:
    - **Gestion de commandes de produits par des clients dans une entreprise**
    - Gestion de livres dans une bibliothèque
- **Remarques:**
  - Ne pas retenir les données calculées ou paramétrées (moyenne, somme, minimum, maximum, etc.).
  - Eliminer les données synonymes (ex. Code Client et n° client)

EXEMPLE DE DICTIONNAIRE DE DONNÉES

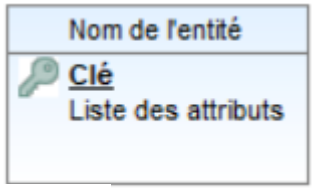
Nom	Description	Type	Taille
Nom_Client	Nom du client	Texte	50
Prenom_Client	Prénom du client	Texte	50
Adresse_Client	Adresse du client	Alphanumérique	255
Ref_Produit	Référence du produit	Entier	
Libelle_Produit	Libellé du produit	Texte	50
Numero_Commande	Numéro de la commande	Entier	
Date_Commande	Date de la commande	Date	
Quantite_Commande	Quantité commandée	Entier	

CONSTRUCTION D'UN MCD: LES ENTITÉS

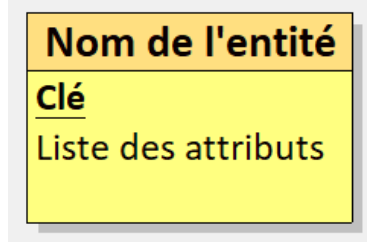
- Dans un MCD ou le modèle entité-association, **une entité** regroupe les données du dictionnaire de données se rapportant au même ensemble.
- Elle est définie comme **un objet** pouvant être identifié distinctement.
- Cet **objet** peut représenter un individu (un client), un objet concret (un livre) ou abstrait (un compte bancaire) ayant une existence propre ou des évènements (une commande).
- Les **entités** sont décrites par **des attributs**, caractéristiques ou propriétés.
- Chaque attribut associe à une entité une valeur dans un domaine (entiers, réels, chaînes de caractères ...)
- Exemples:
  - L'entité « Client » a les attributs : nom, prénom et adresse
  - L'entité « Produit » a les attributs: référence et libellé

IDENTIFIANT ET REPRÉSENTATION GRAPHIQUE D'UNE ENTITÉ

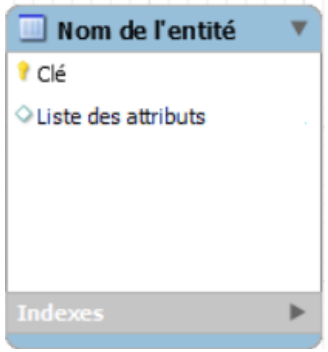
- Un **ensemble d'entités** contient des entités similaires (instances d'entité), mais distinguables deux à deux.
  - Exemples : des personnes, des voitures dans un parking, une bibliothèque de livres...
- Une **clé** ou un **identifiant** est un ensemble minimal d'attributs permettant de déterminer une et une seule entité à l'intérieur de l'ensemble.
- Deux entités différentes doivent avoir des clés différentes.
- Exemple
  - Un numéro de livres caractérise l'exemplaire : c'est une clé[1]
  - Le numéro de la carte nationale d'identité caractérise une personne: c'est une clé.
- **Représentation graphique:**



Logiciel professionnel (avec licence)



Looping freeware (outil pédagogique[1])

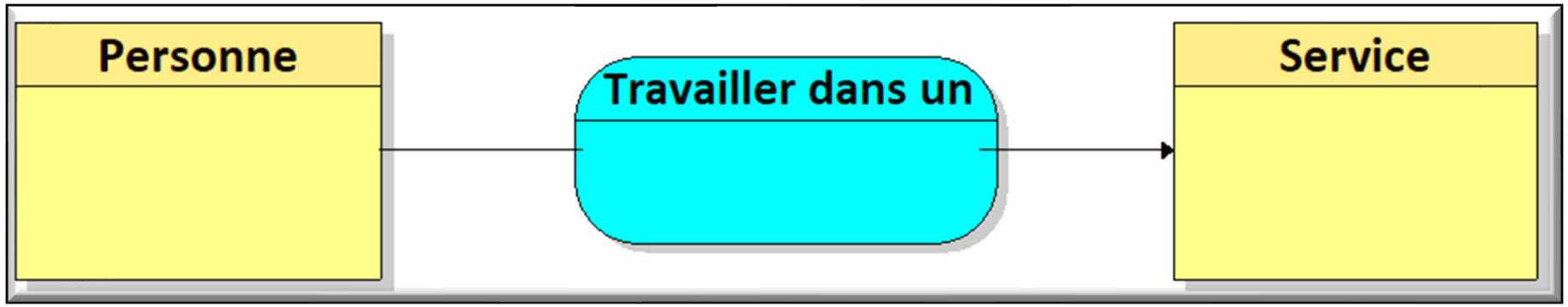


MySQL Workbench freeware (Reverse engineering +éditeur SQL)

[1]TUTORIAL YOUTUBE: HTTPS://WWW.YOUTUBE.COM/WATCH?LOCALE=FR-FR&V=_CPT56TFUEA

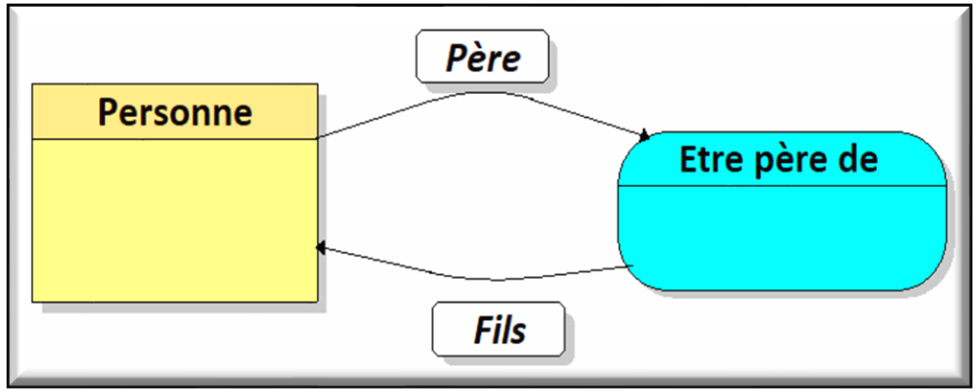
LES ASSOCIATIONS

- On appelle **association** une liste ordonnée d'ensemble d'entités : A = (E1,E2, ... Ek) où k est Le nombre d'entités impliquées dans cette association.
- k est appelé dimension ou **degré** de l'association.
- Les associations sont caractérisées, comme les entités, par un nom (verbe à l'infinitif) et éventuellement des attributs.

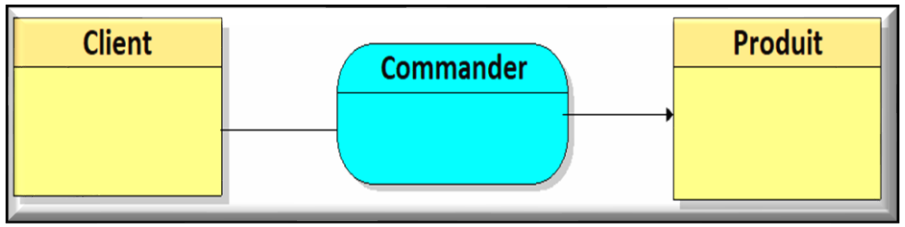


TYPES D'ASSOCIATION

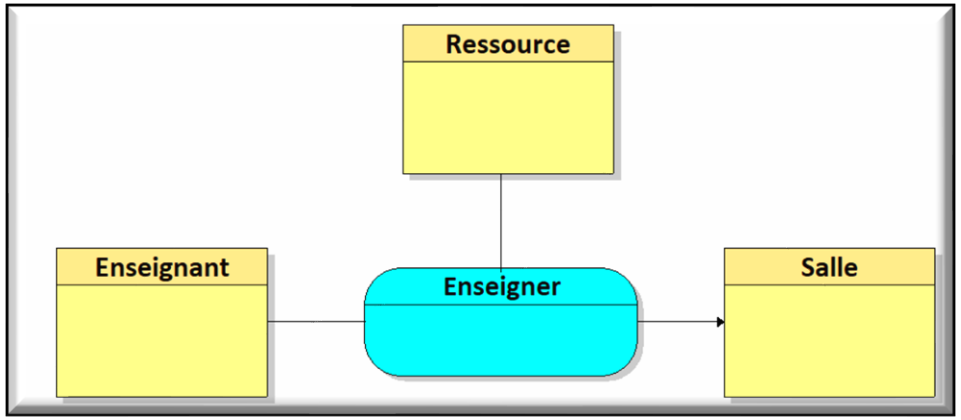
- **Association réflexive :** c'est une association qui relie une entité à elle-même. On peut libeller chaque "patte" selon son rôle dans l'association.



- **Association binaire :** c'est le type standard et qui relie deux entités différentes.

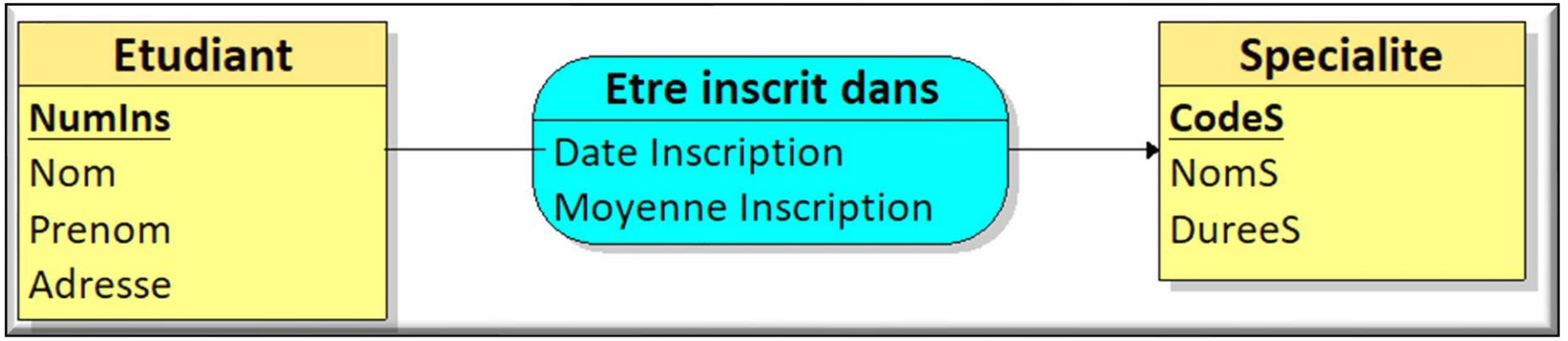


- **Association n-aire** : c'est le cas où une association relie au moins trois entités.



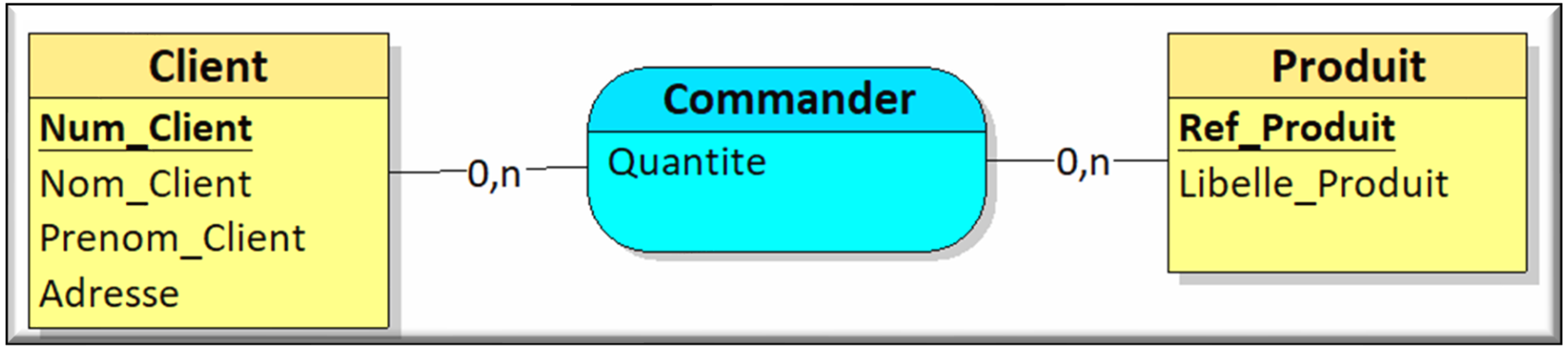
ATTRIBUTS ASSOCIÉS À UNE ASSOCIATION

- Lorsque on a 2 d'entités (e1, e2) et qu'on veut associer à ce couple certaines valeurs, on associe un attribut à cette association.
- Exemple:
  - Les propriétés de l'entité « Etudiant » peuvent être : numéro d'inscription, nom et prénom,
  - Les propriétés de l'entité « Spécialité » peuvent être : le code de la spécialité, le nom et la durée.
  - Les propriétés de de l'association « est inscrit dans » peuvent être : la date d'inscription et la moyenne d'inscription.



LES CARDINALITÉS (1)

- Une cardinalité est un couple de valeurs indiquant les nombres minimum et maximum d'éléments associés.
- C'est les nombre maximum et minimum de liens réels que peut recevoir une instance d'entité pour une association donnée.
- Les cardinalités maximales sont nécessaires pour la création de la base de données.
- Elles traduisent les règles de gestion ou les contraintes (à voir plus loin dans le cours) propres aux entités.
- **Exemple 1:**
  - Un client peut commander de 0 à n produits.
  - Un produit peut être commandé par 0 ou n clients.



LES CARDINALITÉS (2)

- **Exemple 2:**
  - Un film peut avoir un et un seul genre
  - Un genre peut avoir 0 ou plusieurs films.

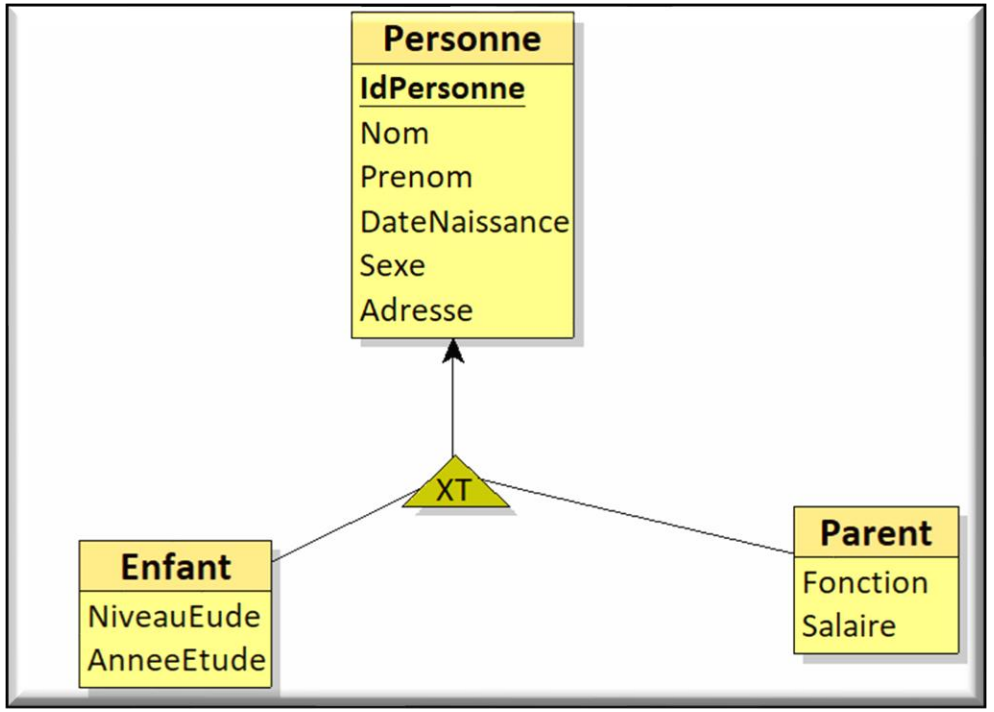


- Cardinalités des associations

Type	E1	E2	Exemple
1:1 (cardinalité maximale de chaque extrémité de l'association)	(0,1) ou (1,1)	(0,1) ou (1,1)	AvoirPourConjoint (Personne)
1:n	(0,1) ou (1,1)	(0,n) ou (1,n)	EtreEditePar (Livre, Editeur)
n:1	(0,n) ou (1,n)	(0,1) ou (1,1)	Editer (Editeur, Livre)
m:n	(0,m) ou (1,m)	(0,n) ou (1,n)	Commander (Client, Produit)

L'HÉRITAGE

- Le MCD définit l'héritage à travers les notions de **généralisation** et de **spécialisation**
- **Généralisation:**
  - Lorsque 2 entités ont des informations communes, ces informations sont regroupées dans une « Entité Mère »
  - Seule l'entité Mère a un identifiant.
  - Les entités « Filles » héritent de tous les attributs de l'entité Mère.
- **Spécialisation:**
  - Les informations spécifiques sont ajoutées dans des « entités Filles »
- L'héritage obéit généralement à deux contraintes d'exclusion et de totalité (Symboles XT sur le MCD).
  - **Contrainte de totalité:** les occurrences doivent appartenir à l'une des 2 entités fille.
  - **Contrainte d'exclusion:** Une occurrence ne peut pas appartenir aux 2 entités filles à la fois.



QUIZ 1

- Rendez-vous sur Microsoft Teams pour le faire le quiz du chapitre I.



10 minutes

LE MODÈLE RELATIONNEL ET LE MODÈLE LOGIQUE DE DONNÉES (MLD)

CHAPITRE II

DÉFINITION DU MODÈLE RELATIONNEL

- Développé en 1970 en utilisant la théorie mathématique des relations.
- Modèle logique orienté enregistrement ou tuple (ligne d'un tableau).
- Constitué de tableaux appelés relations.
- Simple: la relation (ou table) est la seule structure du modèle.
- La définition de relation est plus générale que dans le modèle entité/association.

DÉFINITION D'UNE RELATION (1)

- Elle est représentée par un tableau à 2 dimensions
- Elle est composée d'un nombre fini de colonnes (attributs)
- Chaque attribut possède un nom unique à l'intérieur d'une relation.
- Il ne peut y avoir 2 tuples (enregistrements ou lignes) identiques.

Nom de la relation

Nom de l'attribut

Clients

IdClient	Nom	Prenom	Adresse
101	Dupont	Patrick	Nevers, France
205	Brel	Antonin	Liège, Belgique
302	Smith	Edouard	Londres, Angleterre
128	Almodovar	Pedro	Barcelone, Espagne

Ligne ou tuple

DÉFINITION D'UNE RELATION (2)

- Chaque attribut possède un domaine de valeurs possibles (D_1, D_2 ,..., D_n).
- **Exemples**:
  - Age : âge possible (0-120)
  - IdClient: de 1 à 10000

- Un tuple est composé de n éléments : (v_1, v_2,..., v_n) où v_1 est dans D_1, v_2 est dans D_2, ..., v_n est dans D_n.
- Une table est un ensemble de m **tuples (lignes)** définis dans les domaines (D_1, D_2 ,..., D_n).

BASE DE DONNÉES RELATIONNELLE (BDR)

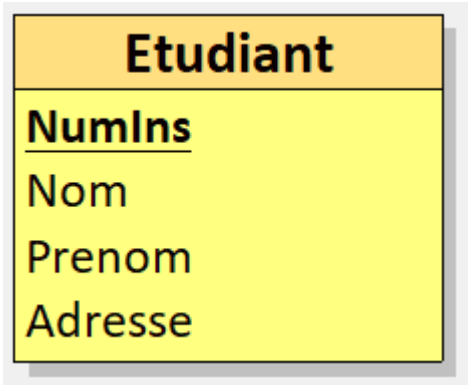
- C'est un regroupement d'un ensemble de relations.
- Chaque relation est nommée de façon unique.
- Elle est représentée par le schéma relationnel
- Un schéma relationnel
  - est composé du nom de la relation R et de la liste de ses attributs (A1, A2, ..., An) .
  - Il est noté R (A1, A2, ..., An)
- Exemple de schéma relationnel:
  - Client (IdClient, Nom, Prenom, Adresse)
- **Attention!** Le schéma relationnel d'une base de données est décrit par plusieurs relations et non une seule.

PASSAGE DU MCD VERS LE MODÈLE LOGIQUE DE DONNÉES (MLD)

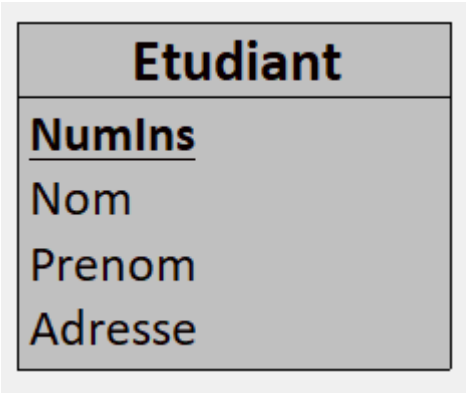
- Pour dériver le modèle relationnel ou le schéma relationnel à partir du modèle conceptuel (MCD), il faut passer par le modèle logique de données (MLD) (cf. slide 9).
- Dans le cas d'une base de données relationnelle, le MLD sera connu sous le nom de MLD-Relationnel ou MLD-R.
- Comme le MCD, le MLD-R est représenté d'une manière graphique en utilisant un certain nombre de règles de passage MCD→MLD-R.
- Le schéma relationnel ne sera autre que la représentation graphique du MLD-R.

RÈGLES DE PASSAGE MCD→MLD-R (1)

- **Règle 1:**
  - Chaque type d'entités devient une relation avec les mêmes attributs
  - Clé ou identifiant d'une entité = la clé primaire de la relation



MCD



MLD-R

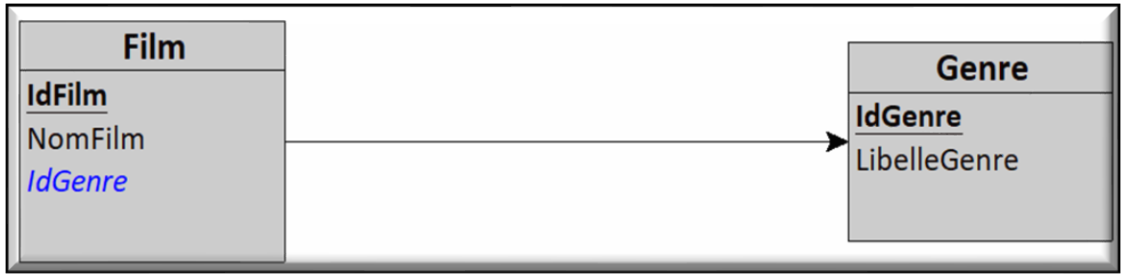
  - Schéma relationnel: Etudiant (**NumIns**, Nom, Prenom, Adresse)

RÈGLES DE PASSAGE MCD→MLD-R (2)

- **Règle 2:**
  - Une association de type 1:n (c'est à dire qui a les cardinalités maximales positionnées à « 1 » d'un côté de l'association et à « n » de l'autre côté) se traduit par la création d'une **clé étrangère** dans la relation correspondante à l'entité côté « 1 ».
  - Cette **clé étrangère** référence la clé primaire de la relation correspondant à l'autre entité.



MCD

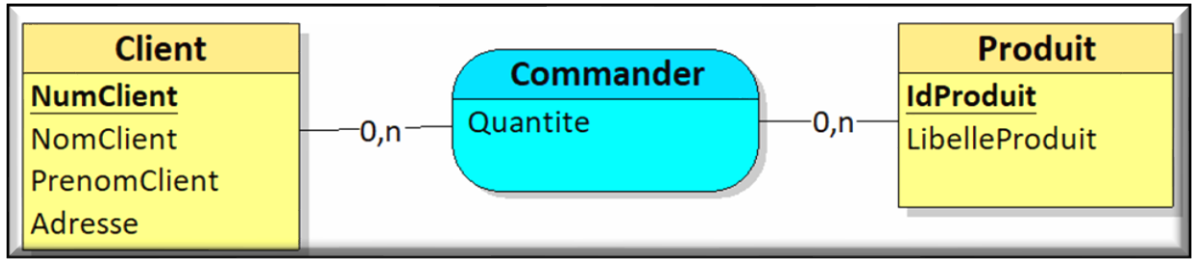


MLD-R

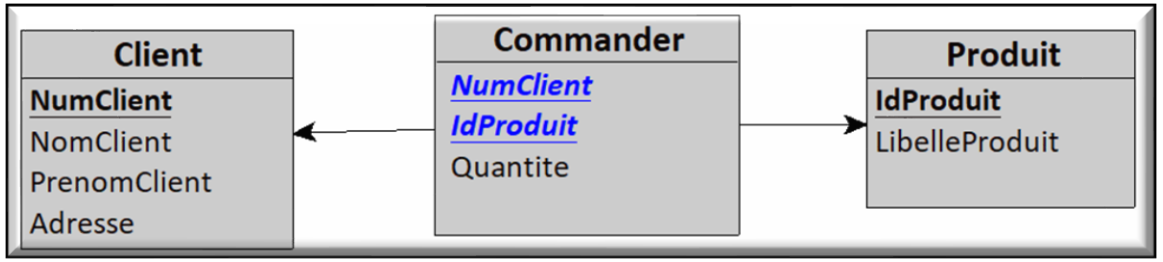
- Schéma relationnel:
  - Genre (**IdGenre**, LibelleGenre)
  - Film (**IdFilm**, NomFilm, #IdGenre). Dans le modèle relationnel, la clé étrangère est précédée par un dièse (#).

RÈGLES DE PASSAGE MCD→MLD-R (3)

- **Règle 3:**
  - Une association de type m:n se traduit par la création d'une table dont la **clé primaire** est composée des **clés étrangères référençant les relations correspondant aux entités liées par l'association**.
  - Les éventuelles propriétés de l'association deviennent des attributs de la relation.



MCD



MLD-R

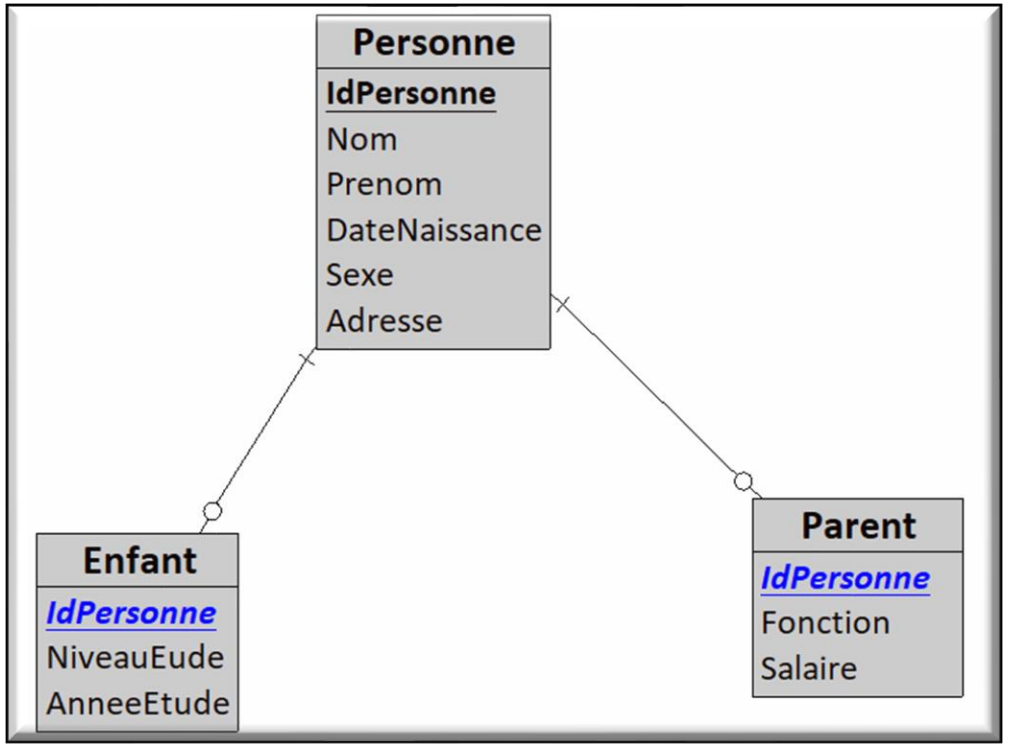
- Schéma relationnel:
  - Client(**NumClient**, NomClient, PrenomClient, Adresse)
  - Produit(**IdProduit**, LibelleProduit,).
  - Commander(**#NumClient**, **#IdProduit**, Quantite).

RÈGLES DE PASSAGE MCD→MLD-R (4)

- **Règle 4:** Toute relation binaire (1,1) est traduite par l'une des solutions suivantes:
  - Fusion des tables des entités qu'elle relie.
  - Report de la clé d'une table dans l'autre table.
- **Règle 5:**
  - **Associations ternaires :** Les règles définies ci-dessus s'appliquent aux associations ternaires.
  - **Associations réflexives :** Les règles définies ci-dessus s'appliquent aux associations réflexives.

RÈGLES DE PASSAGE MCD→MLD-R (5)

- Pour transformer la notion d'héritage en MCD en MLD-R, plusieurs options sont possibles:
  - **Pas de table Mère:**
    - Privilégié lorsqu'il y a très peu d'attributs dans la table Mère
    - La table Mère n'est pas impliquée dans plusieurs associations
  - **Pas de table Fille:**
    - Très peu d'attributs dans les tables Filles.
    - Tous les attributs sont regroupés dans la table Mère.
  - **Présence des tables Mère et Filles (héritage par référence):**
    - Clé primaire de la table Mère devient clé primaire et étrangère dans les tables Filles.



DEGRÉ D'UNE RELATION

- Le degré d'une relation est le nombre d'attributs (n) dans son schéma relationnel
- Exemples:
  - Etudiant (**NumEtudiant**, Nom, Téléphone, Adresse, TéléphoneBureau, Age, DepartementEtude)

    → Cette relation est de degré 7
  - Client (**NumClient**, Nom, Prenom, Adresse)

    → Cette relation est de degré 4
  - Produit (**IdProduit**, LibelleProduit)

    →Cette relation est de degré 2

CONTRAINTES RELATIONNELLES

- Il y a 3 types de contraintes à respecter dans le schéma relationnel
  - Les contraintes de domaines
  - Les contraintes de clés
  - Les contraintes d'intégrité
- Elles doivent être vérifiées par chaque instance du schéma relationnel
- **Contraintes de domaines**
  - La valeur de chaque attribut dans un tuple est atomique (non divisible).
  - La valeur doit respecter le format des données du domaine de l'attribut
    - Type de données: entier, réel, date, caractère
    - Ensemble de valeurs: valeurs positives, intervalle de valeurs, etc.

CONTRAINTES RELATIONNELLES (2)

- **Contraintes de clés**
  - Toute relation doit posséder une clé primaire qui identifie un tuple de façon unique.
  - Une relation peut posséder plusieurs clés candidates. Le cas échéant, l'ensemble de tous les attributs constitue une clé.
- **Contraintes d'intégrité-entité**
  - **Intégrité relation:**
    - Aucune clé primaire ne doit être nulle.
  - **Intégrité référentielle:**
    - Contraintes relatives à l'usage des clés étrangères: une clé étrangère ne peut être référencées dans une relation si elle n'a pas été définie en tant que clé primaire dans une autre relation.

EXERCICE: EXEMPLE COMPLET

- **Enoncé**
  - On veut créer une BD relationnelle pour gérer les commandes des clients. Un client est identifié par son numéro, son nom, son prénom et son adresse. Il peut passer plusieurs commandes à des dates différentes. Un commande peut concerner plusieurs produits. Il convient de préciser la quantité des produits achetés pour chaque commande. Un produit est décrit par son identifiant et son libellé.

- **Questions:**
  1. Dériver le MCD associé à cet énoncé (Identifier les entités, leurs clés et leurs attributs ainsi que les associations et leurs cardinalités).
  2. En déduire le MLD-R et le schéma relationnel de la base.

LE LANGAGE SQL (STRUCTURED QUERY LANGAGE)

CHAPITRE III

INTRODUCTION

- Langage **des SGBD relationnels** inventé à IBM San Jose en1974 (Astrahan & Chamberlin)
- Basé sur le calcul de tuples & algèbre relationnelle
- Evolution contrôlée par l'ANSI[1] (SQL1, 2, 3, 2003, 2008 et 2011)
- SQL 3 (SQL99) : Révision majeure
  - Définition de types, de programmation, de requêtes,
  - Premières capacités orienté objet: accès objet-relationnel (nommage des objets)
- Constitué de 3 parties:
  - Langage de Définition de Données (LDD) : Création, modification, suppression des structures de la BD
  - Langage de Manipulation de Données (LMD) : Ajout, modification, suppression, interrogation des données de la BD
  - Langage de Contrôle de Données (LCD) : Gestion des protections d'accès

[1] AMERICAN NATIONAL STANDARDS INSTITUTE

LDD: CRÉATION D'UNE BASE DE DONNÉES

- Le langage SQL est insensible à la casse (Ne fait pas la différence entre majuscules et minuscules).
- Création de base de données
  - CREATE DATABASE <nom_base>;

```
mysql> CREATE DATABASE MaBase;
Query OK, 1 row affected (0.01 sec)
```

- Création d'une base de données avec l'option « If not exists » permet de ne pas retourner d'erreur si une base du même nom existe déjà.
  - CREATE DATABASE IF NOT EXISTS < nom_base >;
- Afficher les bases de données
  - SHOW DATABASES;

```
mysql> CREATE DATABASE MaBase;
ERROR 1007 (HY000): Ne peut créer la base 'mabase'; elle existe déjà
mysql> CREATE DATABASE IF NOT EXISTS MaBase;
Query OK, 1 row affected, 1 warning (0.00 sec)
```

- Suppression d'une base de données
  - DROP DATABASE <nom_base>;
- Suppression d'une base de données sans afficher un message si elle n'existe pas.
  - DROP DATABASE IF EXISTS <nom_base>;
- Utiliser une base de données
  - USE <nom_base>;

```
mysql> USE MaBase;
Database changed
mysql>
```

LDD: CRÉATION D'UNE TABLE

- **Syntaxe:**

Mot clé

Bloc optionnel

```
CREATE TABLE < nom_table> (
 <nom_attribut1>  <type 1>  [ DEFAULT < valeur_défaut 1> ],
<nom_attribut 2>  <type 2>  [ DEFAULT < valeur_défaut 2> ],
…,
<nom_attribut n>  <type n>  [ DEFAULT < valeur_défaut n> ],
[ CONSTRAINT < nom_contrainte 1>  <contrainte> ],
…,
[ CONSTRAINT < nom_contrainte n>  <contrainte> ]);
```

Bloc optionnel

```
mysql> create table Fournisseur(
    -> NumFournisseur Integer,
    -> NomFournisseur Varchar(50),
    -> PrenomFournisseur Varchar(50),
    -> AdresseFournisseur Varchar(200),
    -> SIREN Integer);
Query OK, 0 rows affected (0.01 sec)
```

TYPES DE DONNÉES SQL (COMPATIBLES MYSQL 8.0)

Types numériques	Types texte	Types date
BOOL/BOOLEAN: valeur booléenne, 0 est faux	CHAR(taille): chaîne de caractères de taille fixe	Date: format YYYY-MM-DD.
SMALLINT: entier court	VARCHAR(taille): chaîne de caractères de taille variable de 0 à 65535	TIME: format hh:mm:ss
INT/INTEGER: entier long	BINARY(taille): stocke des valeurs en binaire	YEAR: année exprimée avec 4 digits.
FLOAT/FLOAT (*taille*, *d*): réel court ayant un nombre de digits égal à taille et un nombre de décimaux égal à d.	TINYTEXT: une chaîne de caractères de longueur maximale égale à 255.	
DOUBLE(*taille, d*)): réel long ayant un nombre de digits égal à taille et un nombre de décimaux égal à d.	ENUM(val1, val2, val3, ...): chaîne de caractères qui a une valeur choisie parmi une liste de valeurs.	

AJOUT DE CONTRAINTE LORS DE LA CRÉATION DE LA TABLE (I)

- Il est possible d'ajouter des contraintes au moment de la création d'une table.

1. **Une de contrainte de clé primaire** : PRIMARY KEY

```
mysql> create table Fournisseur(
    -> NumFournisseur Integer PRIMARY KEY,
    -> NomFournisseur Varchar(50),
    -> PrenomFournisseur Varchar(50),
    -> AdresseFournissuer Varchar(50));
Query OK, 0 rows affected (0.01 sec)
```

2. **Une contrainte de non nullité:** L'attribut ne peut pas prendre de valeur nulle NOT NULL
3. **Une contrainte d'unicité:** UNIQUE

```
mysql> create table Fournisseur(
    -> NumFournisseur Integer PRIMARY KEY,
    -> NomFournisseur Varchar(50) NOT NULL,
    -> PrenomFournisseur Varchar(50),
    -> AdresseFournissuer Varchar(50),
    -> SIREN Integer UNIQUE);
Query OK, 0 rows affected (0.01 sec)
```

AJOUT DE CONTRAINTE LORS DE LA CRÉATION DE LA TABLE (2)

4. **Une contrainte référentielle:** contrainte de clé étrangère REFERENCES

```
mysql> create table Pays(
    -> NomPays Varchar(50) PRIMARY KEY,
    -> Population Integer);
```

Clé étrangère

```
mysql> create table Fournisseur(
    -> NumFournisseur Integer PRIMARY KEY,
    -> NomFournissuer Varchar(50) NOT NULL,
    -> PrenomFournisseur Varchar(50),
    -> AdresseFournissuer Varchar(50) references Pays(NomPays),
    -> SIREN Integer Unique);
Query OK, 0 rows affected (0.01 sec)
```

5. **Une contrainte de domaine:** CHECK

Syntaxe: <attribut> <type> CONSTRAINT < nom_contrainte 1> <contrainte>

```
mysql> create table Pays(
    -> NomPays Varchar(50) Primary key,
    -> Population Integer Constraint Population_CK1 CHECK (Population > 0),
    -> Continent Varchar(50) Constraint Continent_CK2 CHECK (Continent IN ('Europe', 'ASIE')));
Query OK, 0 rows affected (0.01 sec)
```

Nom des contraintes

MODIFICATION DE LA STRUCTURE D'UNE TABLE APRÈS CRÉATION (1)

- Il est possible de modifier la définition (description, structure) d'une table après sa création en utilisant la clause ALTER TABLE
  - En ajoutant une colonne ou une contrainte: ADD
  - En modifiant une colonne ou une contrainte: MODIFY
  - En changeant le nom de la table: RENAME
- Syntaxe

```
ALTER TABLE  <nom-table>
RENAME TO <nouveau_nom_table> |
ADD ( <nom_col 1> <type_col 1> [ DEFAULT <valeur 1>][contraintes_col 1] , …
        <nom_col n> <type_col n> [ DEFAULT <valeur n>][contraintes_col n] ,
          [contrainte_table] )|
MODIFY (<nom_col 1> <type_col 1> [ DEFAULT <valeur 1>][contraintes_col 1]) , …
          <nom_col n> <type_col n> [ DEFAULT <valeur n>][contraintes_col n]) ;
```

MODIFICATION DE LA STRUCTURE D'UNE TABLE APRÈS CRÉATION (2)

- **Exemples:**
  - Renommage d'une table

```
mysql> alter table Fournisseur rename to Fourn;
Query OK, 0 rows affected (0.02 sec)

mysql> describe Fourn;
+--------------------+-------------+------+-----+---------+-------+
| Field              | Type        | Null | Key | Default | Extra |
+--------------------+-------------+------+-----+---------+-------+
| NumFournisseur     | int         | NO   | PRI | NULL    |       |
| NomFournissuer     | varchar(50) | NO   |     | NULL    |       |
| PrenomFounisseur   | varchar(50) | YES  |     | NULL    |       |
| AdresseFournisseur | varchar(50) | YES  |     | NULL    |       |
| SIREN              | int         | YES  | UNI | NULL    |       |
+--------------------+-------------+------+-----+---------+-------+
5 rows in set (0.00 sec)
```

La commande DESCRIBE/DESC <nom_table> permet de décrire le nom de la table

  - Ajout d'une colonne:

```
mysql> ALTER TABLE Fourn ADD TypeF varchar(20);
Query OK, 0 rows affected (0.04 sec)
Enregistrements: 0  Doublons: 0  Avertissements: 0
```

```
mysql> desc Fourn;
+--------------------+-------------+------+-----+---------+-------+
| Field              | Type        | Null | Key | Default | Extra |
+--------------------+-------------+------+-----+---------+-------+
| NumFournisseur     | int         | NO   | PRI | NULL    |       |
| NomFournissuer     | varchar(50) | NO   |     | NULL    |       |
| PrenomFounisseur   | varchar(50) | YES  |     | NULL    |       |
| AdresseFournisseur | varchar(50) | YES  |     | NULL    |       |
| SIREN              | int         | YES  | UNI | NULL    |       |
| TypeF              | varchar(20) | YES  |     | NULL    |       |
+--------------------+-------------+------+-----+---------+-------+
6 rows in set (0.00 sec)
```

Colonne ajoutée

  - Modification d'une colonne

```
mysql> Alter table Fourn MODIFY TypeF varchar(30) DEFAULT 'Externe';
Query OK, 0 rows affected (0.03 sec)
Enregistrements: 0  Doublons: 0  Avertissements: 0
```

SUPPRESSION D'UNE TABLE: DROP TABLE

- Supprimer la table et tout son contenu (structure + données).
- Syntaxe:

  DROP TABLE <nom_table>;

```
mysql> drop table Fourn;
Query OK, 0 rows affected (0.01 sec)

mysql> drop table Pays;
Query OK, 0 rows affected (0.01 sec)
```

- **Remarque !**
  - Avec la clause ALTER TABLE ADD CONSTRAINT <nom_contrainte> , il est possible d'ajouter des contraintes de clé étrangère FOREIGN KEY <nom_attribut> REFERENCES après la création des tables
    ➔ L'ordre de création des tables n'est plus important.

```
mysql> create table Fournisseur (NumFournisseur Integer PRIMARY Key,
    -> NomFournisseur Varchar(50) Not Null,
    -> PrenomFournisseur Varchar(50),
    -> AdresseFournisseur Varchar(50),
    -> SIREN Integer UNIQUE,
    -> TypeF varchar(30));
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> create table Pays(
    -> NomPays Varchar(50) PRIMARY KEY,
    -> Population Integer Constraint Population_CK CHECK (Population >0));
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> Alter table Fournisseur ADD CONSTRAINT Fournisseur_FK FOREIGN KEY(AdresseFournisseur) REFERENCES Pays(NomPays);
Query OK, 0 rows affected (0.05 sec)
```

SUPPRESSION DE COLONNES ET DE CONTRAINTES

- Deux manières sont possibles pour supprimer des contraintes d'une table
  - ALTER TABLE <nom_table> DROP <nom_colonne>;

```
mysql> Alter table Fournisseur DROP COLUMN TypeF;
Query OK, 0 rows affected (0.05 sec)
Enregistrements: 0  Doublons: 0  Avertissements: 0

mysql> desc Fournisseur;
+--------------------+-------------+------+-----+---------+-------+
| Field              | Type        | Null | Key | Default | Extra |
+--------------------+-------------+------+-----+---------+-------+
| NumFournisseur     | int         | NO   | PRI | NULL    |       |
| NomFournissuer     | varchar(50) | NO   |     | NULL    |       |
| PrenomFounisseur   | varchar(50) | YES  |     | NULL    |       |
| AdresseFournisseur | varchar(50) | YES  |     | NULL    |       |
| SIREN              | int         | YES  | UNI | NULL    |       |
+--------------------+-------------+------+-----+---------+-------+
```

- De la même manière, il est possible de supprimer une contrainte comme suit:
  - ALTER TABLE <nom_table> DROP CONSTRAINT <nom_contrainte>;

```
mysql> alter table Pays drop constraint Continent_CK2;
Query OK, 0 rows affected (0.05 sec)
Enregistrements: 0  Doublons: 0  Avertissements: 0

mysql> alter table Pays drop constraint Population_CK1;
Query OK, 0 rows affected (0.03 sec)
Enregistrements: 0  Doublons: 0  Avertissements: 0
```

LANGAGE DE MANIPULATION DE DONNÉES (LMD)

- Il existe 3 commandes principales pour la manipulation des données sous SQL:
  - INSERT INTO: ajouter un tuple dans une table
  - UPDATE: changer des valeurs dans les tuples d'une table
  - DELETE FROM: éliminer les tuples d'une table

LA COMMANDE INSERT

- **Syntaxe**

INSERT INTO <nom_table> [(<nom_colonne 1>, <nom_colonne 2>, …, <nom_colonne n>)] VALUES (<valeur 1>, <valeur 2>, … ,<valeur n>);

```
mysql> insert into PAYS values ('France', 68000000, 'Europe');
Query OK, 1 row affected (0.01 sec)

mysql> insert into PAYS (NomPays, Population) values ('Allemagne', 84000000);
Query OK, 1 row affected (0.00 sec)

mysql> select * from PAYS;
+-----------+------------+-----------+
| NomPays   | Population | Continent |
+-----------+------------+-----------+
| France    |   68000000 | Europe    |
| Allemagne |   84000000 | NULL      |
+-----------+------------+-----------+
2 rows in set (0.00 sec)
```

Insertion terme à terme

LA COMMANDE UPDATE

- La clause UPDATE … SET permet de modifier lignes d'une table.
- Syntaxe

UPDATE <nom_table>

SET <nom_colonne 1> = <valeur 1> |
    <sous_requête 1>,

…,

<nom_colonne n> = <valeur n> | <sous_requête n>

[WHERE <condition>];

```
mysql> update Pays set Population=65000000, Continent='Europe';
Query OK, 2 rows affected (0.01 sec)
Enregistrements correspondants: 2  Modifi├®s: 2  Warnings: 0

mysql> select * from PAYS;
+-----------+------------+-----------+
| NomPays   | Population | Continent |
+-----------+------------+-----------+
| France    |   65000000 | Europe    |
| Allemagne |   65000000 | Europe    |
+-----------+------------+-----------+
2 rows in set (0.00 sec)
```

MAJ des colonnes Population et Contient

Sous requête update avec condition

```
mysql> update Pays set Population=84000000 where NomPays='Allemagne';
Query OK, 1 row affected (0.00 sec)
Enregistrements correspondants: 1  Modifi├®s: 1  Warnings: 0

mysql> select * from PAYS;
+-----------+------------+-----------+
| NomPays   | Population | Continent |
+-----------+------------+-----------+
| France    |   65000000 | Europe    |
| Allemagne |   84000000 | Europe    |
+-----------+------------+-----------+
2 rows in set (0.00 sec)
```

LA COMMANDE DELETE

- La clause DELETE permet de supprimer des lignes d'une table.
- Syntaxe

DELETE FROM <nom_table>

[WHERE <condition>];

```
mysql> delete from Pays where Population >70000000;
Query OK, 1 row affected (0.00 sec)

mysql> select* from Pays;
+---------+------------+-----------+
| NomPays | Population | Continent |
+---------+------------+-----------+
| France  |   65000000 | Europe    |
+---------+------------+-----------+
1 row in set (0.00 sec)
```

Delete from <nom_table> vide complètement la table

```
mysql> delete from Pays;
Query OK, 1 row affected (0.00 sec)

mysql> select* from Pays;
Empty set (0.00 sec)
```

LA REQUÊTE SELECT

- Requête d'interrogation de la base de données
- Syntaxe:

SELECT [DISTINCT] (<nom_attribut 1, …, nom_attribut n>) → liste des attributs de projection

FROM (<nom_table 1>, …, <nom_table n>) → liste des relations touchées par la question

[WHERE (<condition 1|Jointure 1 >, …, < condition n| Jointure n>)] → liste des critères de restriction

[GROUP BY (<nom_attribut 1>, …, <nom_attribut n>)] → liste des attributs d'agrégation/groupement

[HAVING <conditions_sur_le groupe>] → liste des critères de restrictions sur les agrégats/ groupes

[ORDER BY (< nom_attribut 1> [ASC/DESC], …, <nom_attribut n> [ASC/DESC])] ; → liste des attributs de tri du résultat

ECRITURE DES CONDITIONS: OPERATEURS DE COMPARAISON

Symbole	Description	Usage
=	égal	Where (Continent = 'Europe')
<>	différent	Where (NomFounisseur <> 'Dupont')
>	plus grand que	Where (Population > 60000000)
>=	plus grand ou égal	Where (Population >= 61000000)
<	plus petit que	Where (Surface < 500000)
<=	plus petit ou égal	Where (Surface <= 490000)

ECRITURE DES CONDITIONS: OPÉRATEURS LOGIQUES (1)

- Tous les prédicats : **AND**

```
mysql> select * from Pays where continent ='Europe' and Population > 60000000;
+-----------+------------+-----------+---------+----------+
| NomPays   | Population | Continent | Surface | Capitale |
+-----------+------------+-----------+---------+----------+
| France    |   68000000 | Europe    |  551000 | Paris    |
| Allemagne |   83000000 | Europe    |  357000 | Berlin   |
+-----------+------------+-----------+---------+----------+
```

```
mysql> select * from Pays;
+-----------+------------+-----------+---------+-----------+
| NomPays   | Population | Continent | Surface | Capitale  |
+-----------+------------+-----------+---------+-----------+
| France    |   68000000 | Europe    |  551000 | Paris     |
| Allemagne |   83000000 | Europe    |  357000 | Berlin    |
| Japon     |  125000000 | Asie      |  378000 | Tokyo     |
| Etats Unis |  333000000 | Amérique  | 9834000 | Washington |
+-----------+------------+-----------+---------+-----------+
```

- Un des prédicats : **OR**

```
mysql> select * from Pays where continent ='Europe' or continent = 'Asie';
+-----------+------------+-----------+---------+----------+
| NomPays   | Population | Continent | Surface | Capitale |
+-----------+------------+-----------+---------+----------+
| France    |   68000000 | Europe    |  551000 | Paris    |
| Allemagne |   83000000 | Europe    |  357000 | Berlin   |
| Japon     |  125000000 | Asie      |  378000 | Tokyo    |
+-----------+------------+-----------+---------+----------+
```

ECRITURE DES CONDITIONS: OPÉRATEURS LOGIQUES (2)

- Négation de la condition : **NOT**

```
mysql> select * from Pays where Not NomPays='Allemagne';
+------------+------------+-----------+---------+------------+
| NomPays    | Population | Continent | Surface | Capitale   |
+------------+------------+-----------+---------+------------+
| Etats Unis |  333000000 | Amérique  | 9834000 | Washington |
| France     |   68000000 | Europe    |  551000 | Paris      |
| Japon      |  125000000 | Asie      |  378000 | Tokyo      |
+------------+------------+-----------+---------+------------+
```

Combinaison de 2 conditions OR et NOT

- Combinaisons:

```
mysql> select * from Pays where (continent ='Europe' or continent = 'Asie') and (NOT NomPays='Allemagne');
+---------+------------+-----------+---------+----------+
| NomPays | Population | Continent | Surface | Capitale |
+---------+------------+-----------+---------+----------+
| France  |   68000000 | Europe    |  551000 | Paris    |
| Japon   |  125000000 | Asie      |  378000 | Tokyo    |
+---------+------------+-----------+---------+----------+
```

ECRITURE DES CONDITIONS: IN, BETWEEN

- Appartenance/Non appartenance à un ensemble: IN, NOT IN

```
mysql> select * from Pays where Monnaie IN ('Euro', 'Dollar', 'Yen');
+------------+------------+----------+---------+------------+---------+
| NomPays    | Population | Continent | Surface | Capitale   | Monnaie |
+------------+------------+----------+---------+------------+---------+
| France     |   68000000 | Europe    |  551000 | Paris      | Euro    |
| Allemagne  |   83000000 | Europe    |  357000 | Berlin     | Euro    |
| Japon      |  125000000 | Asie      |  378000 | Tokyo      | Yen     |
| Etats Unis |  333000000 | Amérique  | 9834000 | Washington | Dollar  |
+------------+------------+----------+---------+------------+---------+
```

Equivalent à where Monnaie='Euro' or Monnaie='Yen' or Monnaie='Dollar'

- Appartenance/Non appartenance à un intervalle : BETWEEN, NOT BETWEEN:

```
mysql> select * from Pays where Population Between 60000000 and 90000000;
+-----------+------------+-----------+---------+----------+---------+
| NomPays   | Population | Continent | Surface | Capitale | Monnaie |
+-----------+------------+-----------+---------+----------+---------+
| France    |   68000000 | Europe    |  551000 | Paris    | Euro    |
| Allemagne |   83000000 | Europe    |  357000 | Berlin   | Euro    |
+-----------+------------+-----------+---------+----------+---------+
```

Equivalent à Population >= 60000000 And Population <= 90000000

ECRITURE DES CONDITIONS: LIKE, VALEUR NULL

- Condition partielle: LIKE, NOT LIKE
  - % : 0 ou n caractères
  - _ : exactement 1 caractère

```
mysql> select NomPays from Pays where NomPays LIKE '%lande';
+----------+
| NomPays  |
+----------+
| Finlande |
| Irlande  |
| Islande  |
+----------+
3 rows in set (0.00 sec)

mysql> select NomPays from Pays where NomPays LIKE 'I_lande';
+---------+
| NomPays |
+---------+
| Irlande |
| Islande |
+---------+
```

0 à n caractères

1 caractère

- Une valeur inconnue ou non définie: IS NULL

```
mysql> select NomPays, Continent from Pays where (Continent IS NULL);
+----------+-----------+
| NomPays  | Continent |
+----------+-----------+
| Irlande  | NULL      |
| Islande  | NULL      |
| Finlande | NULL      |
+----------+-----------+
```

L'opérateur IS NOT NULL permet d'avoir les pays avec des continents non nuls

BLOCS SELECT IMBRIQUES

- On considère le schéma de BD suivant:
  - Produit (NumP, NomP, CouleurP , PrixP)
  - Fournisseur (NumF, NomF, AdrF)
  - La table association: PF (#NumP, #NumF, Quantite)
- Pour retrouver la couleur et le nom de tous les produits du fournisseur 1, on utilise la **jointure procédurale** (Select Imbriqués)

```
mysql> select * from Produit;
+------+-------------+-------+----------+
| NumP | NomP        | PrixP | CouleurP |
+------+-------------+-------+----------+
|    1 | Marteau     |    30 | rouge    |
|    2 | Vis         |     5 | noir     |
|    3 | Tourne-vis  |    10 | rouge    |
|    4 | Tourne-écrou|    15 | vert     |
+------+-------------+-------+----------+
```

```
mysql> select * from fournisseur;
+------+--------+----------+
| NumF | NomF   | AdresseF |
+------+--------+----------+
|    1 | Dupont | France   |
|    2 | Bernier| France   |
+------+--------+----------+
```

```
mysql> select * from PF;
+------+------+----------+
| NumP | NumF | Quantite |
+------+------+----------+
|    1 |    1 |     1000 |
|    3 |    1 |     2000 |
|    2 |    2 |      500 |
|    4 |    2 |     2000 |
+------+------+----------+
```

Select Interne pour récupérer les numéros de produits de la table association

```
mysql> select NomP, CouleurP from Produit where NumP in (select NumP from PF where NumF=1);
+-----------+----------+
| NomP      | CouleurP |
+-----------+----------+
| Marteau   | rouge    |
| Tourne-vis| rouge    |
+-----------+----------+
2 rows in set (0.00 sec)
```

BLOCS SELECT IMBRIQUES: LA COMMANDE ANY

- La commande ANY permet de comparer une valeur au résultat d'une sous-requête
  - Retourne vrai si au moins une valeur de la sous-requête vérifie la condition.
- Exemples:
  - Numéros des fournisseurs de produits rouges

```
mysql> select NumF from PF where NumP = ANY (select NumP from Produit where CouleurP='rouge');
+------+
| NumF |
+------+
|    1 |
|    1 |
+------+
```

  - Numéro des fournisseurs qui fournissent des produits dont le numéro est inférieur à au moins un des produits coûtant plus que 10 euros.

```
mysql> select NumF, NumP from PF where NumP < Any (Select NumP from Produit where PrixP >10 );
+------+------+
| NumF | NumP |
+------+------+
|    1 |    1 |
|    2 |    2 |
|    1 |    3 |
+------+------+
```

le select interne retourne les numéros de produits 1 et 4

BLOCS SELECT IMBRIQUES: LA COMMANDE ALL

- La commande ALL permet de comparer une valeur au résultat d'une sous-requête
  - Retourne vrai si toutes les valeurs de la sous-requête vérifie la condition.
- Exemple:
  - Numéro des fournisseurs qui fournissent des produits dont le numéro est inférieur à tous les produits coûtant plus que 10 euros.

```
mysql> select NumF, NumP from PF where NumP < all (Select NumP from Produit where PrixP >10 );
Empty set (0.00 sec)
```

Il n'y a pas de produits dont NumP < 1 et NumP < 4

le select interne retourne les numéros de produits 1 et 4

TRAITEMENT DES RÉSULTATS

- Fonctions sur les colonnes
- Élimination de tuples répétés DISTINCT
- Définition de l'ordre des tuples ORDER BY
- Regroupement de résultats GROUP BY

FONCTIONS SUR LES COLONNES

- Attributs calculés

- Opérateurs d'agrégation:
  - Opérateurs sur attributs numériques

    SUM: somme des valeurs des tuples sélectionnés

    AVG : moyenne
  - Opérateurs sur tous types d'attributs

    MIN: minimum

    MAX : maximum

    COUNT: nombre de tuples sélectionnés

```
mysql> select NomPays, Population/Surface from Pays;
+------------+--------------------+
| NomPays    | Population/Surface |
+------------+--------------------+
| France     |           123.4120 |
| Allemagne  |           232.4930 |
| Japon      |           330.6878 |
| Etats Unis |            33.8621 |
| Irlande    |               NULL |
| Islande    |               NULL |
| Finlande   |               NULL |
+------------+--------------------+
```

```
mysql> select min(Population), max(Population), avg(Population), sum(surface), count(*)
 from pays where continent='Europe';
+-----------------+-----------------+-----------------+--------------+----------+
| min(Population) | max(Population) | avg(Population) | sum(surface) | count(*) |
+-----------------+-----------------+-----------------+--------------+----------+
|        68000000 |        83000000 |   75500000.0000 |       908000 |        2 |
+-----------------+-----------------+-----------------+--------------+----------+
```

Sélectionner le min, le max, la moyenne de la population, la somme de la surface et le nombe de tuples qui vérifient la condition

SUPPRESSION DES DOUBLONS ET TRI DES RÉSULTATS

- DISTINCT pour la suppression des doublons

```
mysql> select distinct continent from pays;
+-----------+
| continent |
+-----------+
| Europe    |
| Asie      |
| Amérique  |
| NULL      |
+-----------+
```

- ORDER BY (ASC/DESC) pour trier les tuples

```
mysql> select Continent, NomPays, Population from pays where Population
 >100000000 order by Continent DESC;
+-----------+------------+------------+
| Continent | NomPays    | Population |
+-----------+------------+------------+
| Asie      | Japon      |  125000000 |
| Amérique  | Etats Unis |  333000000 |
+-----------+------------+------------+
```

Par défaut l'ordre de tri est ascendant

```
mysql> select Continent, NomPays, Population from pays where surface
>300000 order by NomPays;
+-----------+------------+------------+
| Continent | NomPays    | Population |
+-----------+------------+------------+
| Europe    | Allemagne  |   83000000 |
| Amérique  | Etats Unis |  333000000 |
| Europe    | France     |   68000000 |
| Asie      | Japon      |  125000000 |
+-----------+------------+------------+
```

LA COMMANDE GROUP BY

- Permet de partitionner l'ensemble des tuples en groupes homogènes suivant un critère.
- Exemple:
  - Avoir pour chaque continent, le minimum, le maximum et la moyenne des populations ainsi que la surface totale et le nombre de pays

```
mysql> select continent, min(Population), max(Population), avg(Population), sum(Surface), count(*) from
 pays group by continent;
+-----------+-----------------+-----------------+-----------------+--------------+----------+
| continent | min(Population) | max(Population) | avg(Population) | sum(Surface) | count(*) |
+-----------+-----------------+-----------------+-----------------+--------------+----------+
| Europe    |        68000000 |        83000000 |   75500000.0000 |       908000 |        2 |
| Asie      |       125000000 |       125000000 |  125000000.0000 |       378000 |        1 |
| Amérique  |       333000000 |       333000000 |  333000000.0000 |      9834000 |        1 |
| NULL      |            NULL |            NULL |            NULL |         NULL |        3 |
+-----------+-----------------+-----------------+-----------------+--------------+----------+
```

Une ligne unique pour chaque groupe

- **Attention!** Chaque attribut dans clause SELECT est:
  - soit inclus dans opérateur d'agrégation
  - soit inclus dans clause GROUP BY

LA CONDITION DE GROUPE HAVING

- Il est possible d'ajouter une condition sur le groupe avec la clause Having associé à GROUP BY
- Exemple:
  - Donner, pour chaque continent ayant une surface supérieure à 2000, le minimum, le maximum et la moyenne des populations ainsi que la surface totale et le nombre de pays

```
mysql> select continent, min(Population), max(Population), avg(Population), sum(Surface), count(*) from
 pays group by continent Having sum(Surface) > 800000;
+-----------+-----------------+-----------------+-----------------+--------------+----------+
| continent | min(Population) | max(Population) | avg(Population) | sum(Surface) | count(*) |
+-----------+-----------------+-----------------+-----------------+--------------+----------+
| Europe    |        68000000 |        83000000 |   75500000.0000 |       908000 |        2 |
| Amérique  |       333000000 |       333000000 |  333000000.0000 |      9834000 |        1 |
+-----------+-----------------+-----------------+-----------------+--------------+----------+
```

- **Attention!** La condition dans HAVING a exactement une valeur pour chaque groupe.

QUELQUES FONCTIONS SQL

- **SUBSTR:** extraire d'une sous-chaîne
  - 3 opérandes: chaîne de caractères, position de la sous sous-chaîne, nombre de caractères à extraire)
  - Exemple: SUBSTR ('abcdefgh', 2, 3) → 'bcd'
- **UPPER**: mettre en majuscule
  - Exemple: UPPER('lili'); → 'LILI'
- **LOWER:** mettre en miniscule
  - Exemple: LOWER ('MARLÈNE'); → 'marlène'

NOMMAGE DES OBJETS EN SQL

- Il est possible d'avoir un accès en objet-relationnel en SQL 3 (créer plusieurs instances de la même table et les référencer par la suite).
- Utiles pour les requêtes de jointure dans l'algèbre relationnelle.

```
mysql> select P1.NomPays, P2.NomPays from Pays P1, Pays P2 where P1.Continent=P2.Continent and NOT
P1.NomPays =P2.NomPays;
+-----------+-----------+
| NomPays   | NomPays   |
+-----------+-----------+
| Allemagne | France    |
| France    | Allemagne |
+-----------+-----------+
```

P1 et P2 sont deux instances du même objet

DÉPENDANCE FONCTIONNELLE ET NORMALISATION

CHAPITRE IV

DÉPENDANCE FONCTIONNELLE: INTRODUCTION

- Exemple de table donnant la liste des commandes des clients:

NumCommande	NomClient	AdresseClient	Montant
1001	Dupont	Nevers, France	200
1005	François	Lille, France	300
1008	Dupont	Nevers, France	500

- Dans cette table, si le client Dupont change d'adresse alors il faudra la changer dans toutes les commandes de même client

➔ **Dépendance entre les attributs NomClient et AdresseClient !**

- Afin de mieux structurer la base de données (intégrité, cohérence et mise à jour), il faut regrouper dans une même relation tous les attributs dépendants les uns des autres

➔ On parle **de dépendance fonctionnelle**

DÉPENDANCE FONCTIONNELLE: DÉFINITION

- Soient X et Y 2 attributs d'une relation R.
- On dit que Y a une dépendance fonctionnelle de X si en connaissant la valeur de X, on peut connaître Y.
- On dit que X détermine Y et on note: X→Y

- Clé d'une relation (Dépendance fonctionnelle)
  - K est une clé candidate de R si K→ R (tous les attributs de la relation)
  - Il n'existe aucun couple de tuples présentant les mêmes valeurs sur l'ensemble d'attributs K.

PROPRIÉTÉS

- Propriétés
  - Transitivité : Si X → Y et Y → Z, alors X → Z
  - Union : Si X → Y et X → Z, alors X → YZ
  - Décomposition: Si X → YZ, alors X → Y et X → Z

- **Attention!** La dépendance fonctionnelle n'est pas commutative

A	B	C	D
A1	B1	C1	D1
A1	B2	C1	D2
A2	B2	C2	D2
A2	B3	C2	D3
A3	B3	C2	D4

On a A→C et pas C→A

NORMALISATION

- Les erreurs de conception peuvent générer de la redondance et l'incohérence de la base de données.
- La normalisation qui s'exécute en plusieurs étapes permet de remédier à ces problèmes.
- Elle s'effectue en s'appuyant sur la dépendance fonctionnelle
- On se limite aux trois premières formes pour la normalisation d'une BD:
  - 1ère forme normale: 1 NF,
  - 2ème forme normale: 2 NF,
  - 3ème forme normale:3 NF

PREMIÈRE FORME NORMALE

- Une relation est en première forme normale si, et seulement si, tout attribut contient une valeur atomique.
- Exemple :
  - Personne (idPersonne, nomPersonne, prenomPersonne, adresse, voitures)
  - Ce schéma de relation n'est pas en 1 NF, car l'attribut adresse est composite (sur plusieurs champs) et l'attribut voitures est multiple. Il faut le décomposer en :
    - Personne(idPersonne, nomPersonne, prenomPersonne, numeroRue, codePostal, Ville)
    - Voiture(idVoiture, modele, marque, #idPersonne)
      - Avec idPersonne est une clé étrangère qui fait référence au schéma relation Personne.

DEUXIÈME FORME NORMALE (1)

- Une relation est en deuxième forme normale si, et seulement si :
  - Elle est en première forme normale ;
  - tout attribut n'appartenant pas à une clé "non clé" ne peut être dépendant de seulement une partie de la clé. Toute la clé est nécessaire pour retrouver l'attribut "non clé"

- Formellement, cette deuxième condition se traduit en:
  - Soit R(X, Y) un schéma de relation (i.e. X→Y) tel que X et Y sont des sous-ensembles d'attributs de R avec $X \cap Y = \emptyset$. Alors, $\forall X' \subset X,\ \ Y' \subseteq Y , X' \rightarrow Y'$ est faux.

DEUXIÈME FORME NORMALE (2)

- Exemple:
  - Affectation (idPersonne, idEtablissement, nomPersonne, prenomPersonne, nomEtablissement).
  - Cette relation n'est pas 2 NF car la clé =(idPersonne, idEtablissement) mais l'attribut nomPersonne ne dépend que de idPersonne.
  - Pour normaliser cette relation, il faut la décomposer de la manière suivante :
  - Personne(idPersonne, nomPersonne, prenomPersonne)
  - Etablissement(idEtablissement, nomEtablissement)
  - Affectation(idPersonne, idEtablissement)
    - Où idPersonne et idEtablissement sont des clés étrangères qui font référence aux relations Personne et Etablissement

TROISIÈME FORME NORMALE

- Une relation est en troisième forme normale si et seulement si :
  - Elle est en deuxième forme normale ;
  - Tout attribut n'appartenant pas à une clé ne dépend pas d'un attribut non-clé.
- Formellement :
  - Soit R(X,Y) un schéma de relation, (i.e. X→Y) tel que X et Y sont des sous-ensembles d'attributs de R avec $X \cap Y = \emptyset$. Alors, $\forall Yi \subseteq Y$, $Yj \subseteq Y$, $Yi \rightarrow Yj$ est faux.

TROISIÈME FORME NORMALE (2)

- Exemple :
  - Etudiant(idEtudiant, nomEtudiant, prenomEtudiant, Etablissement, villeEtablissement).
- Cette relation n'est pas 3 NF, car l'attribut villeEtablissement dépend de l'attribut établissement qui n'est pas une clé candidate.
- Pour normaliser cette relation, il faut la décomposer de la manière suivante :
  - Etudiant(idEtudiant, nomEtudiant, prénomEtudiant, #idEtablissement)
  - Etablissement(idEtablissement, nomEtablissement, villeEtablissement)
    - Où idEtablissement est une clé étrangère qui fait référence au schéma Etablissement.

ALGÈBRE RELATIONNELLE

CHAPITRE V

INTRODUCTION

- Le premier langage étudié dans ce cours est l'algèbre relationnelle.
- Il consiste en un ensemble d'opérations qui permettent de manipuler des relations, considérées comme des ensembles de tuples.
- On peut ainsi faire l'union ou la différence de deux relations, sélectionner une partie de la relation, effectuer des produits cartésiens ou des projections, etc.
- On distingue les opérateurs unaires et les opérateurs binaires.

OPÉRATEURS UNAIRES: LA SÉLECTION

- Soit $R(a_1, a_2, ..., a_N)$ une relation.
- L'opérateur de sélection noté σ permet de sélectionner un ensemble de tuples de R qui satisfont une certaine condition noté $\sigma_{condition}$ (R) (opérateur SQL, SELECT).

Client

RefCl	Nom	Adresse
200	Dupont	Nevers, France
300	François	Lille, France
400	Dupont	Nevers, France

Produit

RefProduit	Libelle
1010	Smartphone
1020	Ordinateur
1070	Imprimante

Commande

RefCl	RefProduit	Quantite
200	1010	50
300	1020	20
400	1070	5
300	1070	3

$\sigma_{Adresse='Nevers, France'}$ (Client).

300	François	Lille, France
400	Dupont	Nevers, France

$\sigma_{RefProduit > 1010}$ (Produit).

1020	Ordinateur
1070	Imprimante

OPÉRATEURS UNAIRES: LA PROJECTION

- La projection permet d'extraire un ensemble d'attributs parmi tous les attributs de R en éliminant les doublons.
- Elle est notée $\pi_{a_1,\ldots,a_k}(R)$
- <u>Exemples:</u>
  - $\pi_{Nom,\ Adresse}(Client)$

Nom	Adresse
Dupont	Nevers, France
François	Lille, France
Dupont	Nevers, France

$\pi_{Libelle}(Produit)$

Libelle
Smartphone
Ordinateur
Imprimante

OPÉRATEURS BINAIRES: UNION

- L'union de deux relations R et S notée (R ∪ S) est une relation qui contient à la fois tous les tuples de R et de S en éliminant les doublons.
- **Exemple**: Soient 2 relations clients français et espagnols ayant la même structure. Un client franco-espagnol a 2 adresses une Espagne et une en France.

RefCl	Nom	Adresse
200	Dupond	Paris, France
300	François	Lille, France
400	Dupont	Nevers, France
550	Vincent-Miguel	Barcelone, Espagne

ClientF

RefCl	Nom	Adresse
350	Almodovar	Madrid, Espagne
550	Vincent-Miguel	Barcelone, Espagne
725	Rodriguez	Cordoue, Espagne

ClientE

ClientF ∪ ClientE

RefCl	Nom	Adresse
200	Dupond	Paris, France
300	François	Lille, France
400	Dupont	Nevers, France
350	Almodovar	Madrid, Espagne
550	Vincent-Miguel	Barcelone, Espagne
725	Rodriguez	Cordoue, Espagne

OPÉRATEURS BINAIRES: INTERSECTION

- L'intersection définit une relation constituée de l'ensemble de tous les tuples présents à la fois dans *R* et dans *S* notée par: **R ∩ S.**
- **Exemple:** ClientF ∩ ClientE

550	**Vincent-Miguel**	**Barcelone, Espagne**

- La différence d'ensemble définit une relation qui comporte les tuples qui existent dans la relation *R* et non dans la relation *S* notée par: **R - S.**
- **Exemple:** R=(ClientF ∪ ClientE), S= ClientF, R-S≠ClientE

350	**Almodovar**	**Madrid, Espagne**
725	Rodriguez	Cordoue, Espagne

- L'union et l'intersection sont associatives et commutatives mais la différence ne l'est pas.

OPÉRATEUR PRODUIT CARTÉSIEN

- Soient deux relations R_1 (A_1, …,A_n) et S_1 (B_1, …,B_n), deux relations. Le produit cartésien Q (A_1, …,A_n, B_1, … B_n) de R par S noté RxS est définie tel que (A_1, …,A_n) appartient à R et (B_1, …,B_n) appartient à S.
- **Exemple**

A_1	A_2	A_3
a_{11}	a_{12}	a_{13}
a_{21}	a_{22}	a_{23}

B_1	B_2
b_{11}	B_{12}
b_{21}	b_{22}

A_1	A_2	A_3	B_1	B_2
a_{11}	a_{12}	a_{13}	b_{11}	b_{12}
a_{11}	a_{12}	a_{13}	b_{21}	b_{22}
a_{21}	a_{22}	a_{23}	b_{11}	b_{12}
a_{21}	a_{22}	a_{23}	b_{21}	b_{22}

- Le produit cartésien est associatif et commutatif.
- Degré (RxS)=Degré(R) + Degré(S)

JOINTURE

- La thêta-jointure définit une relation qui contient les tuples qui satisfont le produit cartésien de R et S sous une condition P, notée $R\theta_P S$ (la condition est vraie dans les 2 relations, **Join** en sql).
- La condition P s'écrit $R.a_i \ op \ S.b_j$, où op est l'un des opérateurs de comparaison ($<, \leq, >, \geq, =, /=$).
- Si l'opérateur de comparaison est =, on parle alors d'équijointure.
- **Exemple: Client $\theta_{Client.RefCl=Commande.RefCl}$ Commande ➔ Opérateur condition '=' ➔ Equijointure.**

RefCl	Nom	Adresse	RefCl	RefProduit	Quantite
200	Dupond	Paris, France	200	1010	50
300	François	Lille, France	300	1020	20
400	Dupont	Nevers, France	400	1070	5
300	François	Lille, France	300	1070	3

JOINTURE NATURELLE

- La jointure naturelle est une équijointure où les attributs communs en doublons dans les 2 relations sont supprimés.
- **Exemple**: **Client** $\theta_{Client.RefCl=Commande.RefCl}$ **Commande** ⟺ **Client*Commande**

RefCl	Nom	Adresse	RefCl	RefProduit	Quantite
200	Dupond	Paris, France	200	1010	50
300	François	Lille, France	300	1020	20
400	Dupont	Nevers, France	400	1070	5
300	François	Lille, France	300	1070	3

- **Exercice:** en utilisant les bons opérateurs
  - Donner le nom des clients, ainsi que les références des produits achetés et les quantités.
  - Pour le client Dupont, donner les libellés de tous les produits qu'il a acheté.

DIVISION

- L'opérateur de division est utilisé pour répondre à une question du genre « Quels sont les produits achetés par tous les clients? ».
- Il est noté par: R÷S

